00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D

ALGORİTMA VE PROGRAMLAMA – II

HAFTA 11

Graf Algoritmalarına Giriş

Temsil, Genişlik ve Derinlik Öncelikli Arama

MODÜL IV · GRAFLAR VE AÇGÖZLÜ STRATEJİ

Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Bahar

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1 Doğruluk ve Ölçüm	**H2** Asimptotik Analiz	**H3** Özyineleme ve Geri İzleme	**H4** Arama Algoritmaları	**H5** Temel Sıralama	**H6** Böl ve Fethet	**H7** Öbek ve Doğrusal Sıralama
0x00	0x01	0x02	0x03	0x04	0x05	0x06
H8 Dinamik Dizi ve Bağlı Liste	**H9** Yığın ve Kuyruk	**H10** Karma Tabloları	**H11** **Graflar: BFS ve DFS**	**H12** Açgözlü Algoritmalar	**H13** Dinamik Programlama	**H14** Sentez ve Final Projesi
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 (teslim: H8 başı) · Final projesi: H14

TAHMİN ET

Geçen haftalardan dört soru (yanıtlar notlarda)

- **S1 (H10).** n = 10^5 öğrenci numarasında yinelenenleri karma kümesi kaç yoklamayla bulur? Bütün çiftleri karşılaştırmak kaç karşılaştırma?
- **S2 (H10).** Python'da `list` bir `set` öğesi olabilir mi? Izgara hücresini (satır, sütun) ziyaret kümesine nasıl koyarız?
- **S3 (H9).** Kuyrukla düzey düzey işlemede, 1'den başlayıp x'in ardılları 2x ve 2x + 1 (≤ 15) iken 3. düzeyde hangi sayılar işlenir?
- **S4 (AP-1 H11).** 5 × 5 ızgarada köşe, kenar ve iç hücrenin kaç 4-komşusu, kaç 8-komşusu var?

HAFTANIN BÜYÜK SORUSU

Büyük Soru: labirent, yol ağı, sosyal ağ, önkoşul listesi

2 ↔ 14

açık odada en kısa yol: BFS ↔ H3 geri izlemesinin ilk yolu (adım; Örnek 10)

10 GB

$V = 10^5$ seyrek grafta komşuluk matrisi (liste 5,60 MB)

108

dokuz derslik önkoşul DAG'ında geçerli ders sırası (Örnek 11)

- Bir labirentten çıkışın en kısa yolu nasıl bulunur ve bir navigasyon uygulaması, bir sosyal ağ ve bir ders önkoşul listesi neden aynı soyut yapıyla modellenebilir?

GERÇEK DÜNYADAN

1735 · 1736 · 1741

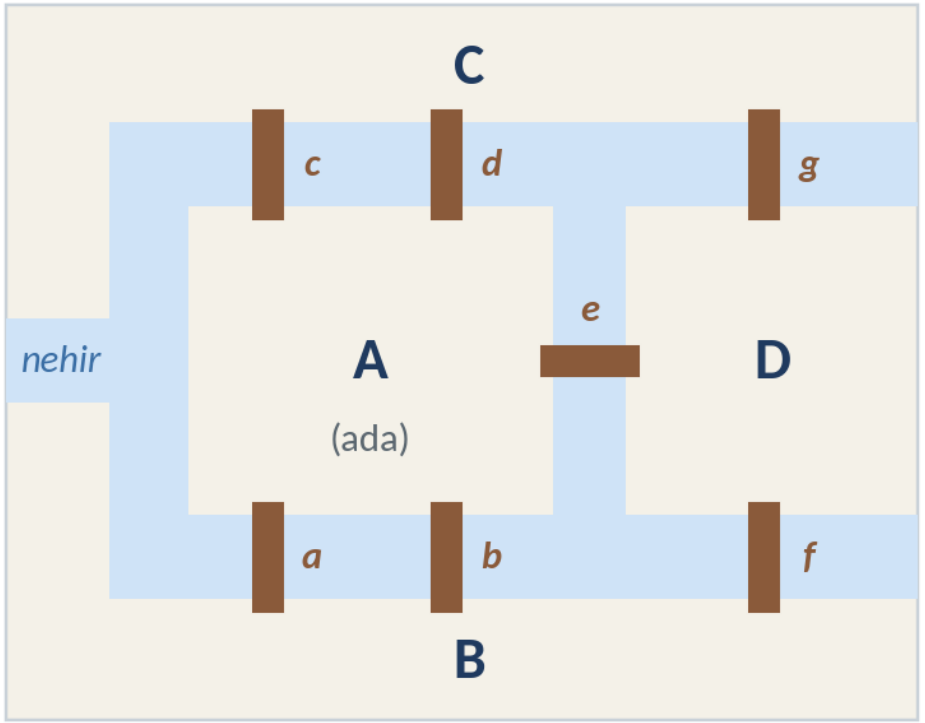
Königsberg'in yedi köprüsü: bir bulmacadan graf kuramına

- Pregel'in kolları kenti **dört kara parçasına** ayırır, **yedi köprü** onları bağlar: her köprüden tam bir kez geçen bir yürüyüş var mı?
- Euler kara parçalarını A–D, köprüleri a–g diye adlandırır: A adasına beş, B, C ve D'ye üçer köprü.
- Akıl yürütme bir **derece** sayımıdır: başlangıç ve bitiş dışında her kara parçasının köprü sayısı çift olmalı; dördü de tek → yürüyüş yok.
- Makale St. Petersburg Akademisi'ne **26 Ağustos 1735**'te sunuldu; **1736** tarihli 8. ciltte yer aldı, cilt **1741**'de basıldı.

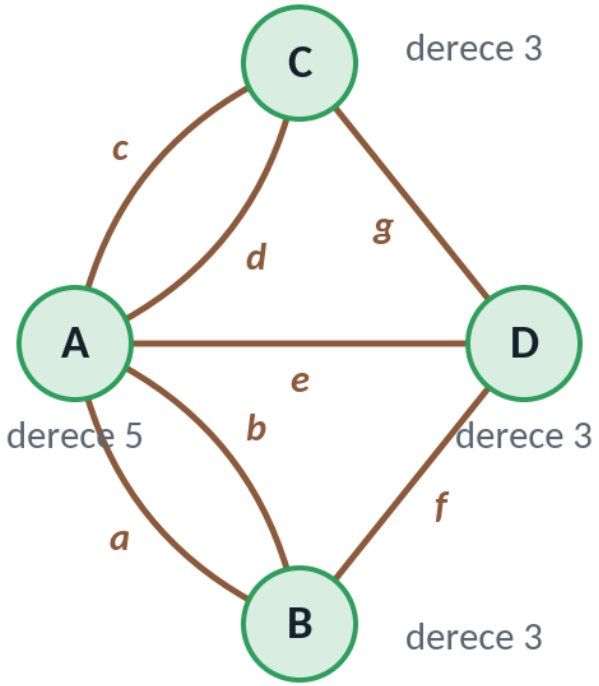
Euler, L. (1741). Solutio problematis ad geometriam situs pertinentis. Commentarii Academiae Scientiarum Petropolitanae, 8, 128–140 · Eneström (1910–1913).

Graf soyutlaması: köşe = kara parçası, kenar = köprü

Kent (şematik): 4 kara parçası, 7 köprü



Graf: köşe = kara, kenar = köprü



dört köşe de tek dereceli →
her köprüden bir kez geçen yürüyüş yok

Euler'in harfleriyle: A'nın derecesi 5, B, C, D'nin 3 — dört tek dereceli köşe, her köprüden bir kez geçen yürüyüş yok

GERÇEK DÜNYADAN

1959 · 1961

Labirentten devre kartına: dalga dalga yayılma

- Edward F. Moore, “The shortest path through a maze” (labirentte en kısa yol): Harvard'daki bir anahtarlama kuramı sempozyumunun bildiriler kitabı, **1959**.
- Genişlik öncelikli aramanın “kaynaktan dalga dalga yayılma” fikri bu labirent sorusuyla birlikte anılır.
- C. Y. Lee (Bell Laboratuvarları, **1961**): devre kartında bağlantı yolu; “bir kaynaktan genişleyen dalgaların bilgisayar modeli”, IBM 704'te; en kısa yol örneklerinde Moore'un algoritmalarından birini kullandığını yazar.
- Örnek 10'un ızgara BFS'si aynı dalgadır: H3 labirentinde 22 adımlık yol, 61 hücre işlenir.

Moore, E. F. (1959). The shortest path through a maze · Lee, C. Y. (1961). IRE Transactions on Electronic Computers, EC-10(3), 346–365.

ÖĞRENME ÇIKTILARI

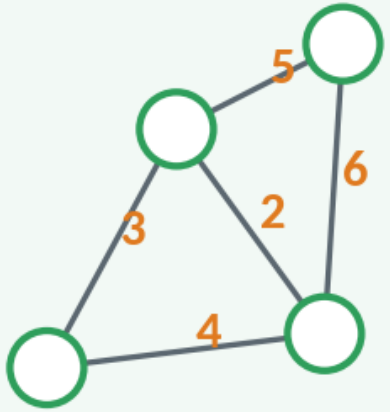
Bu hafta sonunda neler yapabileceksiniz?

ÖÇ 11.1	Gerçek dünya problemlerini graf olarak modeller (köşe, kenar, yön, ağırlık); komşuluk matrisi ve komşuluk listesi temsillerini karşılaştırır.
ÖÇ 11.2	Genişlik öncelikli aramayı (BFS) kuyrukla gerçekler; ağırlıksız graflarda en kısa yol uzaklıklarını ve yolun kendisini üretir.
ÖÇ 11.3	Derinlik öncelikli aramayı (DFS) özyinelemeli ve açık yığınla gerçekler; bağlı bileşenleri ve döngüleri bulur.
ÖÇ 11.4	Yönlü döngüsüz graflarda (DAG) topolojik sıralamayı Kahn algoritması ve DFS ile yapar.
ÖÇ 11.5	Izgara problemlerini (labirent, taşkın doldurma, ada sayma) örtük graf olarak çözer; BFS ve DFS'nin O(V + E) karmaşıklığını açıklar.

Modellemek üç karardır: köşe, kenar, yön ve ağırlık

Yol ağı

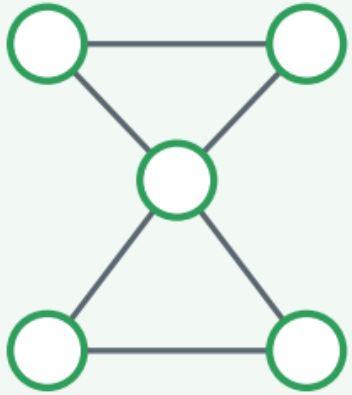
kavşak · yol



ağırlıklı, yönsüz

Sosyal ağ

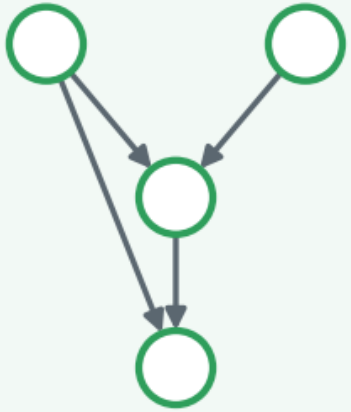
kişi · tanışıklık



yönsüz

Önkoşul

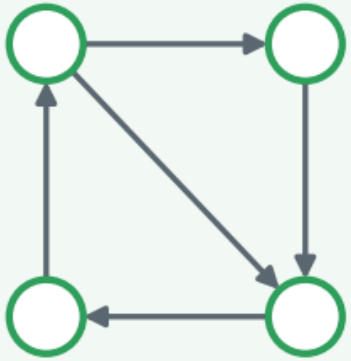
ders · önkoşul



yönlü, döngüsüz

Web

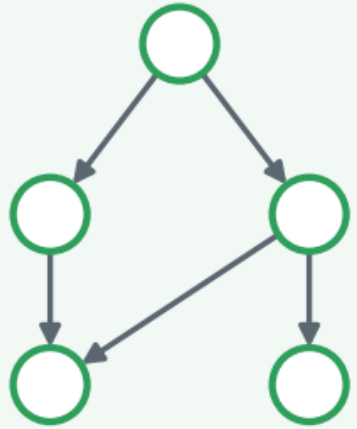
sayfa · bağlantı



yönlü, döngülü

Oyun

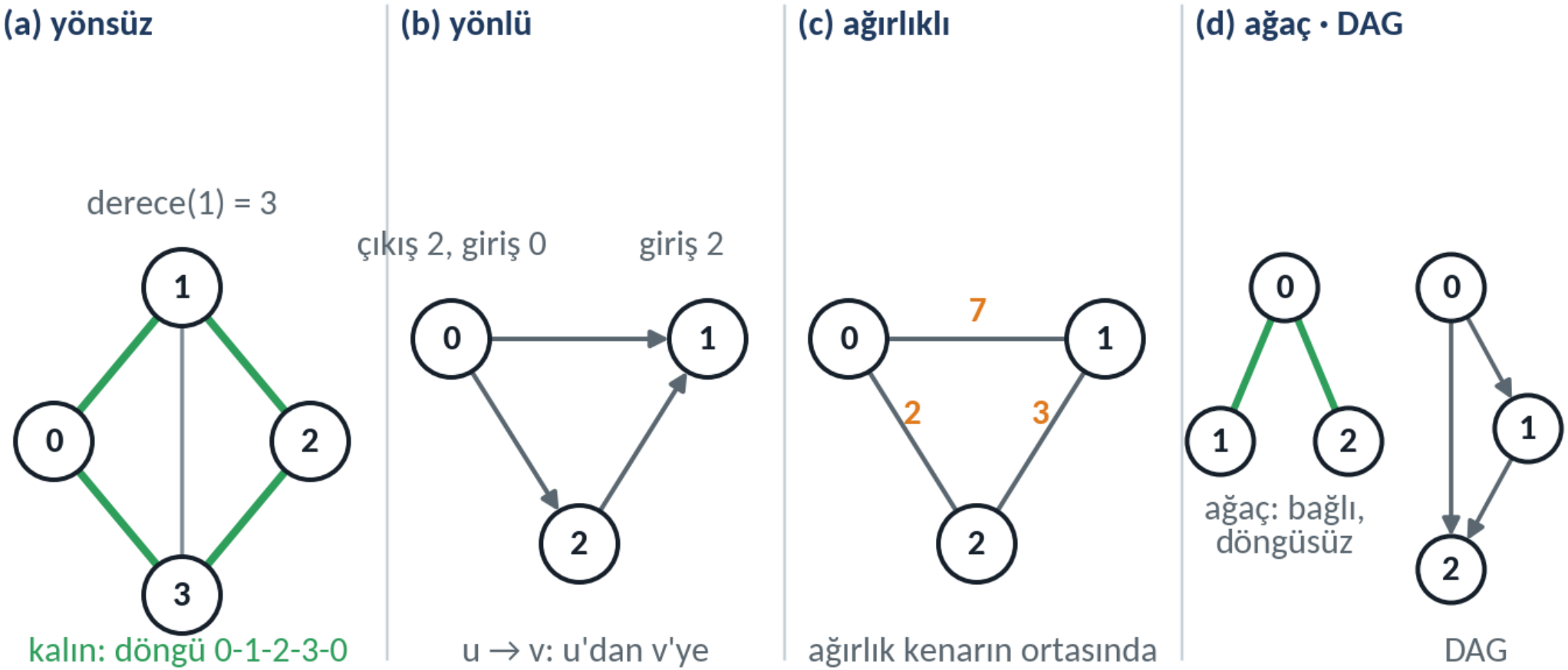
durum · hamle



yönlü (durum uzayı)

Yol ağı (ağırlıklı), sosyal ağ (yönsüz), önkoşul (yönlü, döngüsüz), web (döngülü), oyun (durum uzayı)

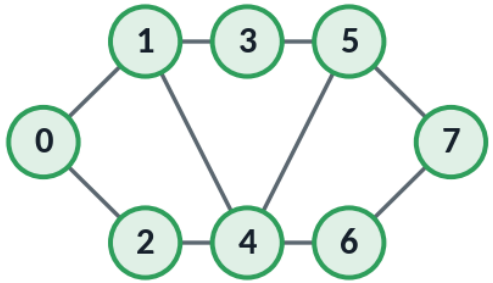
Temel terimler: derece, yol, döngü, bağlılık, ağaç, DAG



Σ derece = 2E (G: 20 = 20); yönlüde Σ çıkış = Σ giriş = E · ağaç: bağlı, döngüsüz, V − 1 kenar

Aynı graf, üç temsil: kenar listesi, matris, liste

G: V = 8, E = 10 (yönsüz)



kenar listesi (dosya sırası)

```
(4,5)  (0,2)  (6,7)
(1,3)  (0,1)  (4,6)
(2,4)  (3,5)  (5,7)
(1,4)
```

komşuluk matrisi (V^2 hücre)

	0	1	2	3	4	5	6	7
0	0	**1**	**1**	0	0	0	0	0
1	**1**	0	0	**1**	**1**	0	0	0
2	**1**	0	0	0	**1**	0	0	0
3	0	**1**	0	0	0	**1**	0	0
4	0	**1**	**1**	0	0	**1**	**1**	0
5	0	0	0	**1**	**1**	0	0	**1**
6	0	0	0	0	**1**	0	0	**1**
7	0	0	0	0	0	**1**	**1**	0

`satır 4 = 01100110`

simetrik: m[u][v] = m[v][u]

komşuluk listesi (artan)

0	1 → 2
1	0 → 3 → 4
2	0 → 4
3	1 → 5
4	1 → 2 → 5 → 6
5	3 → 4 → 7
6	4 → 7
7	5 → 6

öğe sayısı: matris $V^2 = 64$ · liste $V + 2E = 28$ · kenar listesi $2E = 20$

G (V = 8, E = 10; Örnek 1 ve Örnek 2): matris $V^2 = 64$ hücre, liste $V + 2E = 28$ öğe, kenar listesi $2E = 20$ sayı

Üç temsilin maliyetleri (V köşe, E kenar)

Matriste u'nun komşuları = u satırının tamamı: Örnek 1 bütün komşuları bulurken $V^2 = 64$ hücre okur (komşu sayısından bağımsız).

İşlem	Kenar listesi	Komşuluk matrisi	Komşuluk listesi
Bellek	Θ(E)	Θ(V^2)	Θ(V + E)
u–v kenarı var mı?	Θ(E)	Θ(1)	Θ(derece(u))
u'nun komşularını dolaş	Θ(E)	Θ(V)	Θ(derece(u))
Bütün grafı dolaş (BFS/DFS)	—	Θ(V^2)	Θ(V + E)
Uygun olduğu yer	dosya, giriş/çıkış; H12 Kruskal	yoğun, küçük V	seyrek, büyük graflar

TAHMİN ET

$V = 10^5$ köşeli seyrek bir sosyal ağ ($E = 4V$). Komşuluk matrisi hücre başına 1 bayt tutsa ne kadar bellek ister?

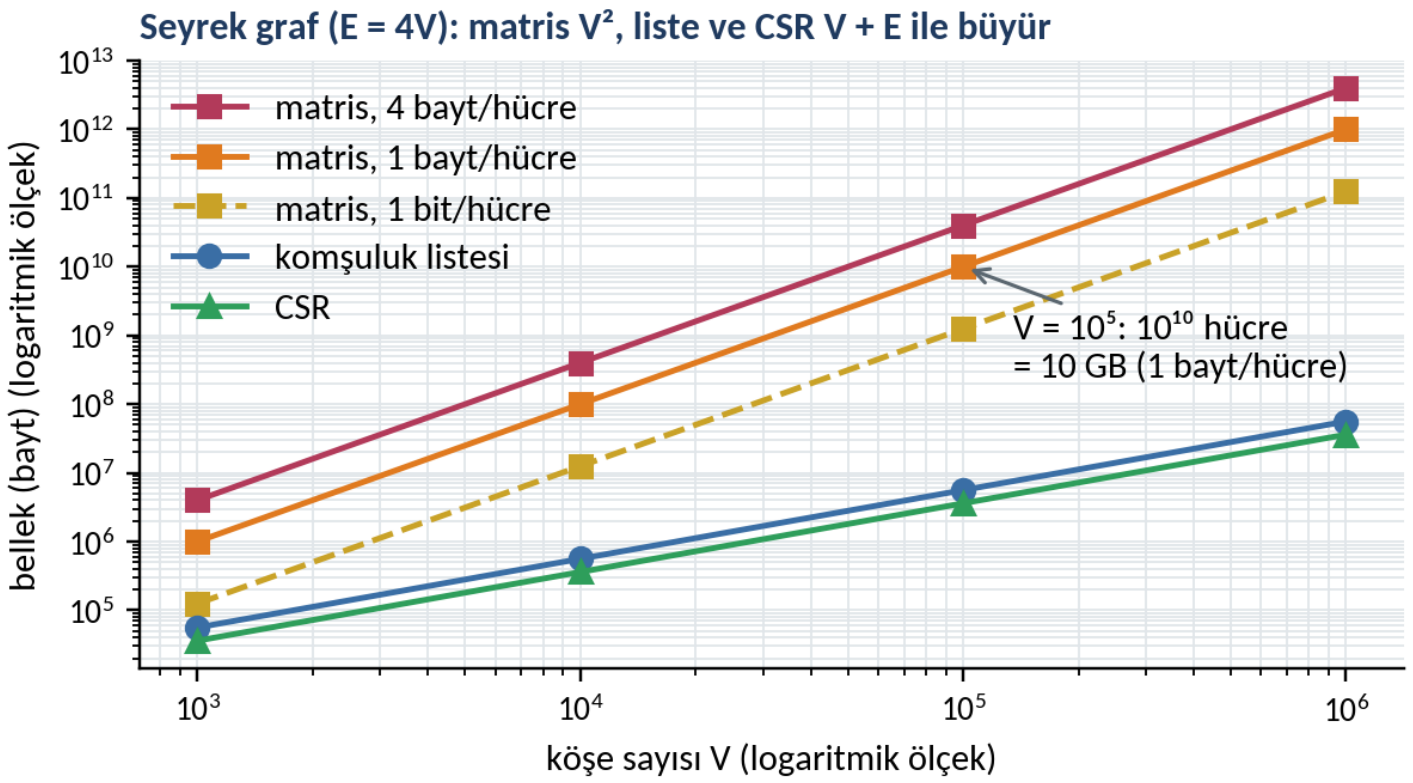
A ≈ 10 MB

B ≈ 100 MB

C ≈ 1 GB

D ≈ 10 GB

Kaputun Altında: matriste bellek patlaması



Seyrek grafta (E = 4V) temsil belleği, hesap (Kavram 1); log–log

- **V = 10^5:** $V^2 = 10^{10}$ hücre → matris 10 GB (1 bayt) · 40 GB (int); liste 5,60 MB, CSR 3,60 MB.
- Matris **V^2** ile, liste ve CSR **V + E** ile büyür.
- Deneyde matris seyrek grafta $V = 2 \cdot 10^4$'ten sonra 1,5 GB'lık bütçeyi aştı.
- **Python:** `[0] * 1000` satırı 8056 bayt (hücre başına ≈ 8,06), `bytearray(1000)` 1057 bayt (hücre başına ≈ 1,06): hücre bir başvurudur.

Üç dilde: matriste u'nun komşuları (satır taraması)

H11_01_komsuluk_matrisi.c — C

```c
int k = 0;
for (int v = 0; v < V; v++) {
    okunan_hucre++;
    if (matris[u][v] != 0) {
        komsu[k++] = v;
    }
}
return k;
```

H11_01_komsuluk_matrisi.cpp — C++

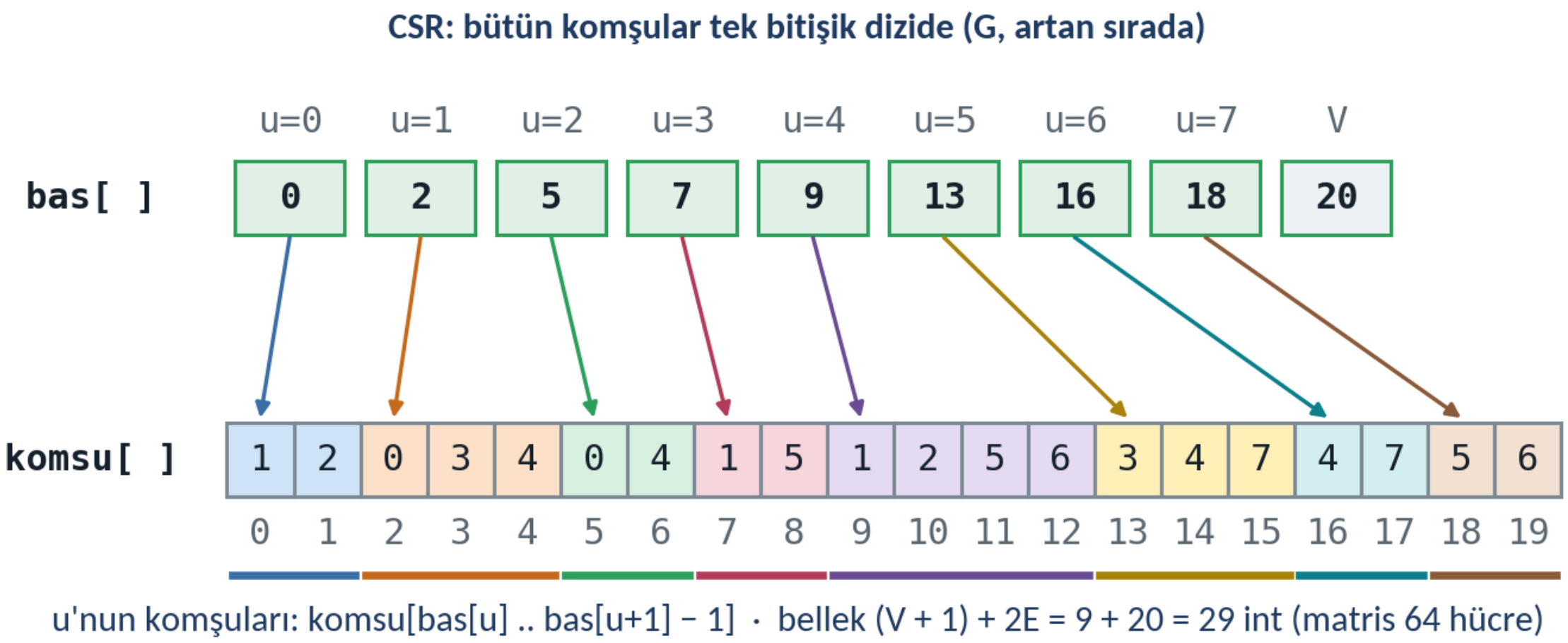
```cpp
std::vector<int> komsu;
int V = static_cast<int>(matris.size());
for (int v = 0; v < V; v++) {
    okunan_hucre++;
    if (matris[u][v] != 0) {
        komsu.push_back(v);
    }
}
return komsu;
```

H11_01_komsuluk_matrisi.py — Python

```python
"""u'nun komşuları: satır u baştan sona taranır (V hücre)"""
global okunan_hucre
komsu = []
for v in range(len(matris)):
    okunan_hucre += 1
    if matris[u][v] != 0:
        komsu.append(v)
return komsu
```

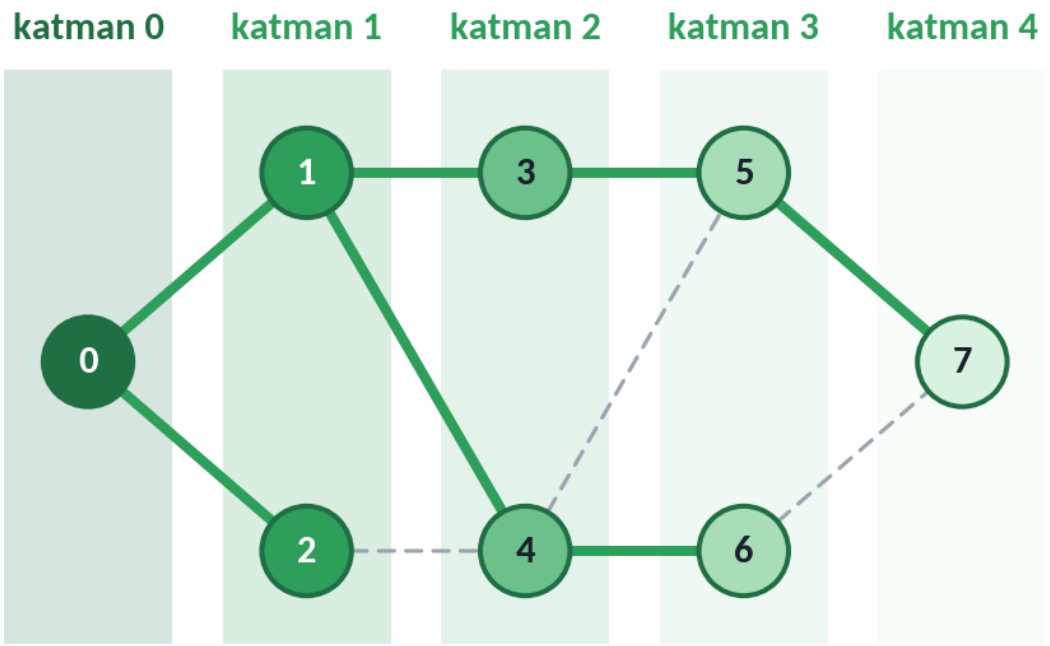
Komşu sayısı ne olursa olsun V hücre okunur; C'de sabit 2B dizi, C++'ta vector<vector<int>>, Python'da liste listesi (H11_01_komsuluk_matrisi; satırlar: C 78–85 · C++ 78–86 · Python 52–59)

Merak: CSR — bütün komşular tek bitişik dizide



u'nun komşuları komsu[bas[u] .. bas[u+1] − 1] · G'de bas = 0 2 5 7 9 13 16 18 20 · deneyde C, V = 10^5: CSR listeden 1,58 kat hızlı

BFS: suya atılan taş gibi, katman katman



kalın: BFS ağacı (ebeveyn → çocuk) · kesikli: ağaç dışı kenar

```
ebeveyn = [-1 0 0 1 1 3 4 5]
```

kuyruk (baş solda)

adım	çıkan	
başla		0
1	0	1 2
2	1	2 3 4
3	2	3 4
4	3	4 5
5	4	5 6
6	5	6 7
7	6	7
8	7	boş

ekleme 8, inceleme 20 (= 2E)

G'de BFS(0) (Örnek 3): katmanlar [0] [1 2] [3 4] [5 6] [7] · kuyruk (baş solda) · kalın: BFS ağacı

TAHMİN ET

G'de BFS(0), kuyruk [4 5] (baş solda). Kuyruktan sıradaki köşe hangisi ve kuyruk ne olur?

A 5 çıkar (en son eklenen); kuyruk [4 7]

B 4 çıkar; yalnız 6 eklenir: [5 6]

C 4 çıkar; 1, 2, 6 eklenir: [5 1 2 6]

D 4 çıkar; 5 ve 6 eklenir: [5 5 6]

BFS: sözde kod, değişmez ve maliyet

```
1  algoritma BFS(G, s)
2    her v için: uzak[v] ← −1; ebeveyn[v] ← −1
3    uzak[s] ← 0;  EKLE(Q, s)
4    iken Q boş değil
5      u ← ÇIKAR(Q)                  ▷ baştan
6      u'nun her v komşusu için (artan)
7        eğer uzak[v] = −1 ise
8          uzak[v] ← uzak[u] + 1
9          ebeveyn[v] ← u
10         EKLE(Q, v)                ▷ işaret EKLERKEN
```

- **Katman değişmezi:** kuyruk her an [d, …, d, d + 1, …, d + 1]; giren v'nin uzak[v]'si kesindir.
- **Maliyet:** her köşe bir kez girer, her liste bir kez taranır: **Θ(V + E)**; matrisle Θ(V^2).
- **Yol:** ebeveyn zincirini t'den s'ye izle, ters çevir; uzunluk = uzak[t].
- Yalnız **ağırlıksız** graflarda en kısa yol (ağırlıkta Dijkstra, H12).

C: BFS ana döngüsü (Örnek 4)

c/H11_04_bfs_en_kisa_yol.c · satır 203–214

C

```c
    q[son++] = s;
    while (bas < son) {
        int u = q[bas++];
        for (size_t i = 0; i < g->komsu[u].boyut; i++) {
            int v = g->komsu[u].veri[i];
            if (uzak[v] == -1) {
                uzak[v] = uzak[u] + 1;
                ebeveyn[v] = u;          /* v'ye ilk u'dan ulaşıldı */
                q[son++] = v;
            }
        }
    }
```

ÖRNEK 4 · G, KAYNAK 0

```
köşe  uzaklık  ebeveyn  en kısa yol
   5        3        3  0 -> 1 -> 3 -> 5
   7        4        5  0 -> 1 -> 3 -> 5 -> 7
kâhin (kenar koşulu + ebeveyn tutarlılığı): geçti
```

TAHMİN ET

G'de BFS(0) sonunda ebeveyn = [-1 0 0 1 1 3 4 5] (–1: kök). Ebeveyn dizisinden çıkarılan 0 → 7 yolu hangisi?

A 7 → 5 → 3 → 1 → 0 (ters çevirmeden)

B 0 → 2 → 4 → 6 → 7

C 0 → 1 → 3 → 5 → 7

D 0 → 1 → 3 → 5 → 4 → 6 → 7

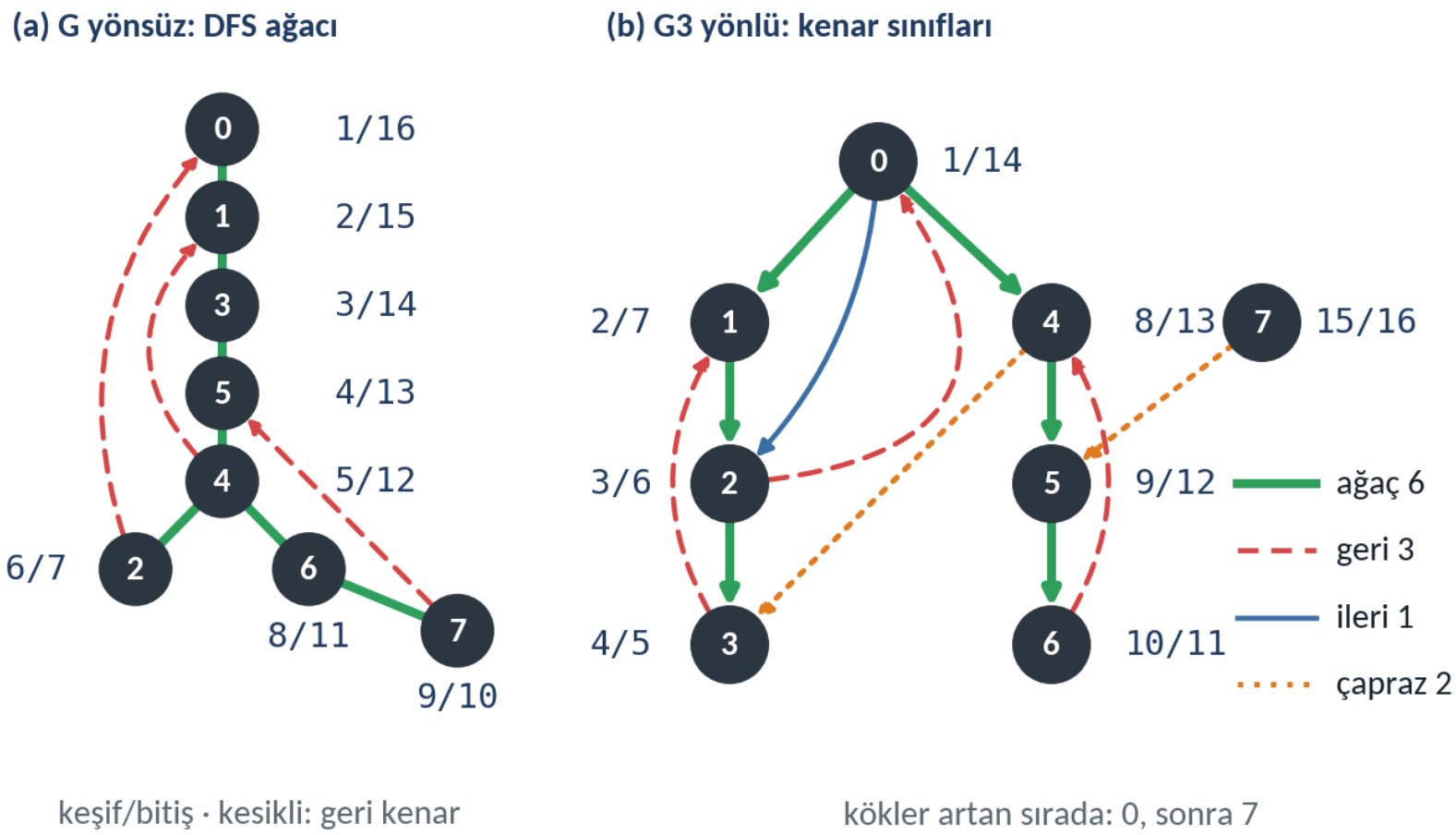
ALGORİTMA KİMLİK KARTI

Genişlik öncelikli arama (BFS) *breadth-first search (BFS)*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Kuyrukla katman katman; işaret eklerken	Kaynak s; ağırlıksız	Θ(V + E)	Θ(V + E)	Θ(V + E)	O(V)

Özellikler	Ağırlıksız en kısa yol ve katmanlar; G'de 8 ekleme, 20 kenar inceleme (= 2E)
Değişmez / doğruluk	Kuyrukta d'lerin ardından yalnız d + 1'ler; giren v'nin uzak[v]'si kesin
Kütüphane karşılığı	C: elle (kuyruk H9) · C++: Boost.Graph breadth_first_search · Python: networkx (yalnız kâhin)
Ne zaman kullanılır	Ağırlıksız en kısa yol, katmanlar, taşkın doldurma, iki parçalılık
Ne zaman kullanılmaz	Kenarlar ağırlıklıysa (Dijkstra, H12)

DFS: önce derine — keşif/bitiş zamanları ve kenar türleri



keşif/bitiş · kesikli: geri kenar

kökler artan sırada: 0, sonra 7

Yönsüz G: parantez (0 (1 (3 (5 (4 (2 2) (6 (7 7) 6) 4) 5) 3) 1) 0) · yönlü G3: ağaç 6, geri 3, ileri 1, çapraz 2

TAHMİN ET

Yönlü G3'te DFS(0): 0 → 1 → 2 → 3 inildi; 3, 2, 1 bitti (bitiş[3] = 5). Sonra 0'dan 4'e inildi (keşif[4] = 8). 4'ün ilk komşusu 3: 4 → 3 hangi kenar?

A ağaç kenarı

B geri kenar (döngü)

C ileri kenar

D çapraz kenar

DFS: sözde kod, üç renk ve parantez yapısı

```
1  algoritma ZİYARET(u)
2    renk[u] ← GRİ;  keşif[u] ← ++zaman
3    u'nun her v komşusu için (artan)
4      eğer renk[v] = BEYAZ ise
5        ebeveyn[v] ← u;  ZİYARET(v)   ▷ ağaç
6      değilse eğer renk[v] = GRİ ise
7        geri kenar: döngü
8      değilse  ileri ya da çapraz
9    renk[u] ← SİYAH;  bitiş[u] ← ++zaman
10 ▷ DFS(G): her beyaz u için (artan) ZİYARET(u)
```

- **Gri değişmezi:** gri köşeler kökten şimdiki köşeye giden yoldur (özyineleme yığını).
- **Parantez teoremi:** iki köşenin [keşif, bitiş] aralıkları ya iç içe ya ayrık; v, u'nun torunu ⇔ aralığı u'nunkinin içinde.
- **Θ(V + E)**; ek bellek O(V), ama özyineleme derinliği V'ye ulaşabilir.

C: özyinelemeli DFS ve kenar sınıflaması (Örnek 5)

c/H11_05_dfs_ozyinelemeli.c · satır 221–235

C

```c
    for (size_t i = 0; i < g->komsu[u].boyut; i++) {
        int v = g->komsu[u].veri[i];
        if (d->renk[v] == BEYAZ) {
            kenar_kaydet(d, u, v, AGAC);
            d->ebeveyn[v] = u;
            dfs_ziyaret(g, v, d);
        } else if (g->yonlu) {
            kenar_kaydet(d, u, v, d->renk[v] == GRI ? GERI :
                                  d->kesif[u] < d->kesif[v] ? ILERI : CAPRAZ);
        } else if (d->renk[v] == GRI && v != d->ebeveyn[u]) {
            kenar_kaydet(d, u, v, GERI);  /* yönsüz: atasına kenar */
        }
    }
    d->derinlik--;
    d->renk[u] = SIYAH;
```

ÖRNEK 5 · YÖNLÜ G3

```
parantez yapısı: (0 (1 (2 (3 3) 2) 1) (4 (5 (6 6) 5) 4) 0) (7 7)
sayılar: ağaç 6, geri 3, ileri 1, çapraz 2 (toplam 12 = E)
```

Üç dilde: kenar sınıfı v'nin rengine göre

H11_06_dfs_acik_yigin.c — C

```c
if (d->renk[v] == BEYAZ) {
    return AGAC;
}
if (d->renk[v] == GRI) {
    return GERI;
}
return d->kesif[u] < d->kesif[v] ? ILERI : CAPRAZ;
```

H11_06_dfs_acik_yigin.cpp — C++

```cpp
if (d.renk[v] == BEYAZ) {
    return AGAC;
}
if (d.renk[v] == GRI) {
    return GERI;
}
return d.kesif[u] < d.kesif[v] ? ILERI : CAPRAZ;
```

H11_06_dfs_acik_yigin.py — Python

```python
"""(u, v) kenarının sınıfı; v'nin rengi değişmeden ÖNCE çağrılır"""
if d.renk[v] == BEYAZ:
    return AGAC
if d.renk[v] == GRI:
    return GERI
return ILERI if d.kesif[u] < d.kesif[v] else CAPRAZ
```

Beyaz → ağaç, gri → geri, siyah → keşif zamanına göre ileri ya da çapraz; C'de Dfs yapısına işaretçi, C++'ta başvuru, Python'da nesne (H11_06_dfs_acik_yigin; satırlar: C 236–242 · C++ 174–180 · Python 129–134)

ALGORİTMA KİMLİK KARTI

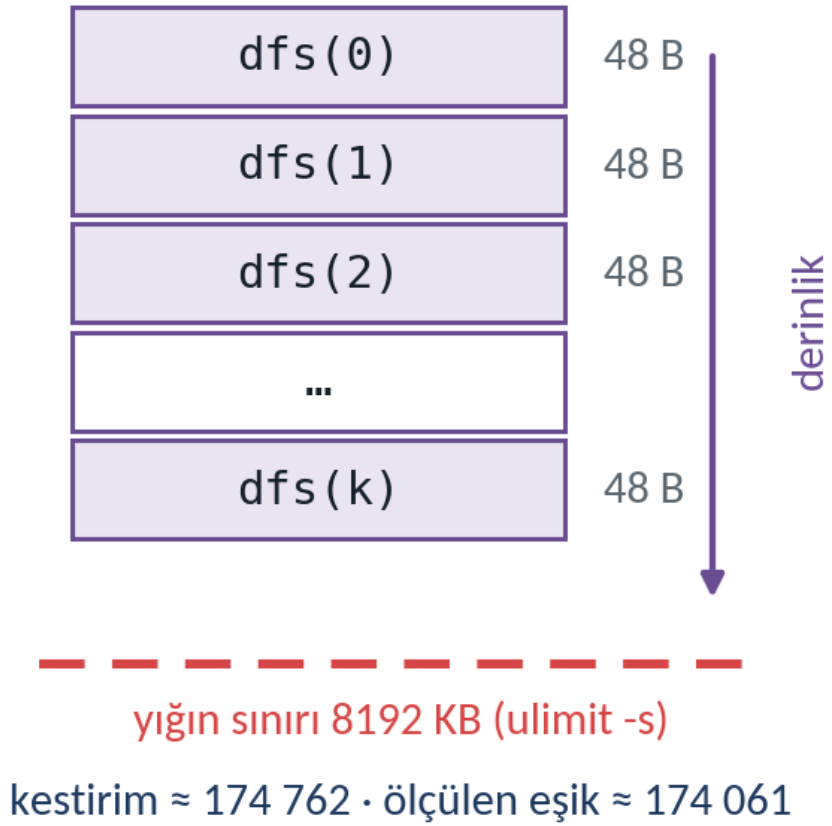
Derinlik öncelikli arama (DFS) *depth-first search (DFS)*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Önce derine; üç renk, keşif/bitiş	Yok (her bileşenden başlanır)	Θ(V + E)	Θ(V + E)	Θ(V + E)	O(V)

Özellikler	G3: ağaç 6, geri 3, ileri 1, çapraz 2; geri kenar = döngü
Değişmez / doğruluk	Gri köşeler kökten şimdiki köşeye giden yol; parantez teoremi
Kütüphane karşılığı	C: elle · C++: Boost.Graph depth_first_search · Python: networkx (yalnız kâhin)
Ne zaman kullanılır	Döngü, bileşen, topolojik sıra, labirent/arama, geri izleme
Ne zaman kullanılmaz	En kısa yol (BFS); derin girdide özyinelemeli sürüm (açık yığın)

Kaputun Altında: derin DFS'de yığın taşması

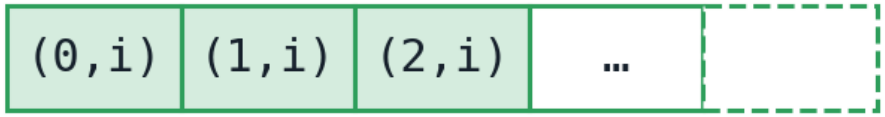
(a) özyinelemeli DFS: çağrı yığını



kestirim ≈ 174 762 · ölçülen eşik ≈ 174 061

(b) açık yığın: çerçeveler öbekte

çerçeve = (köşe, sıradaki komşu indisi)



dizi malloc ile V çerçevelik ayrılır

derinlik yalnız bellekle sınırlı

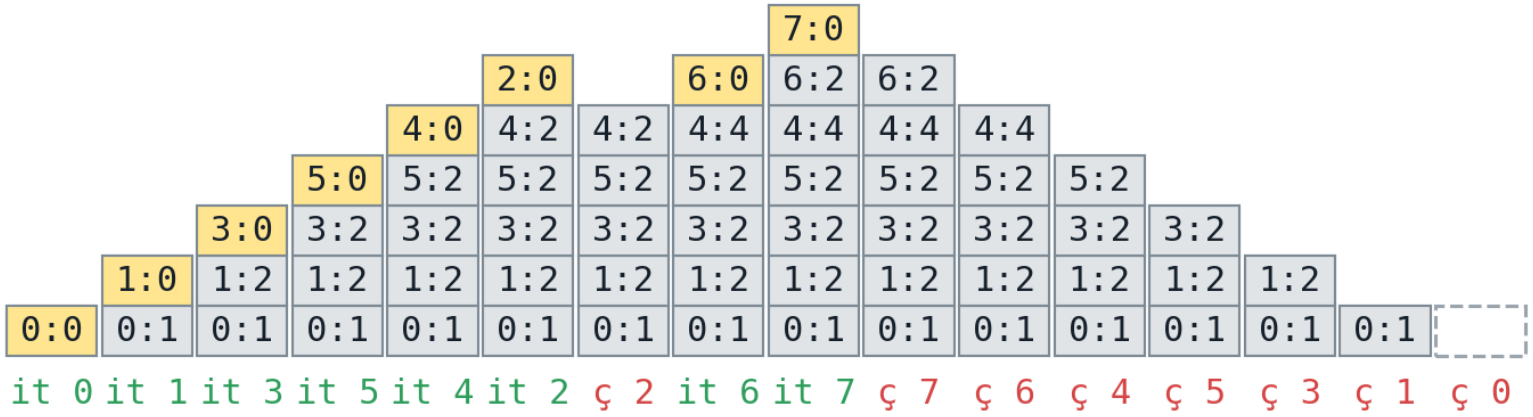
deney: n = 10^6 yol grafı, C 9,28 ms, çökme yok

Python özyineleme sınırı: 1000 çağrı

Çerçeve 48 B, yığın 8 MiB: eşik ≈ 174 061 köşe (C -O0, ölçülen) · Python 995 çağrıda RecursionError

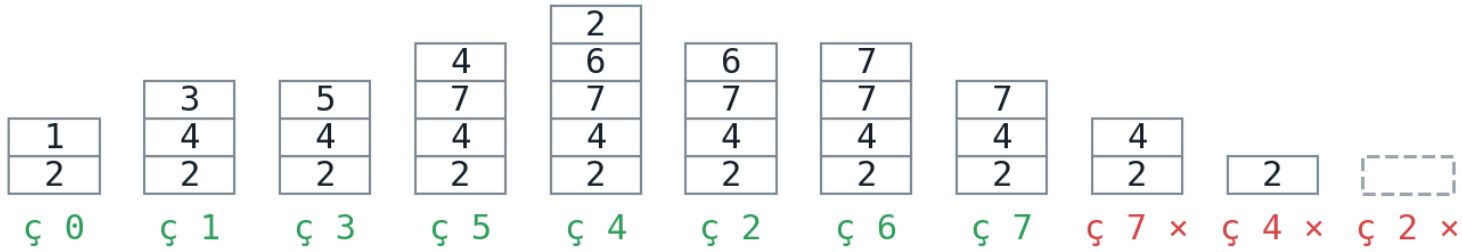
Açık yığınlı DFS: çerçeve = (köşe, sıradaki komşu)

(A) çerçeve yığını: (köşe:sıradaki komşu indisi) — özyinelemeli DFS ile aynı keşif/bitiş



it u: u keşfedildi (gri) · ç u: u'nun komşusu kalmadı, bitti (siyah)

(B) ters sırada itme (ç u: çıkar, komşuları ters sırada it); itme 11, atlanan 3 (×)



(A) çerçeve yığını özyinelemeyle aynı keşif/bitiş · (B) ters sırada itme: ziyaret 0 1 3 5 4 2 6 7, itme 11, atlanan 3

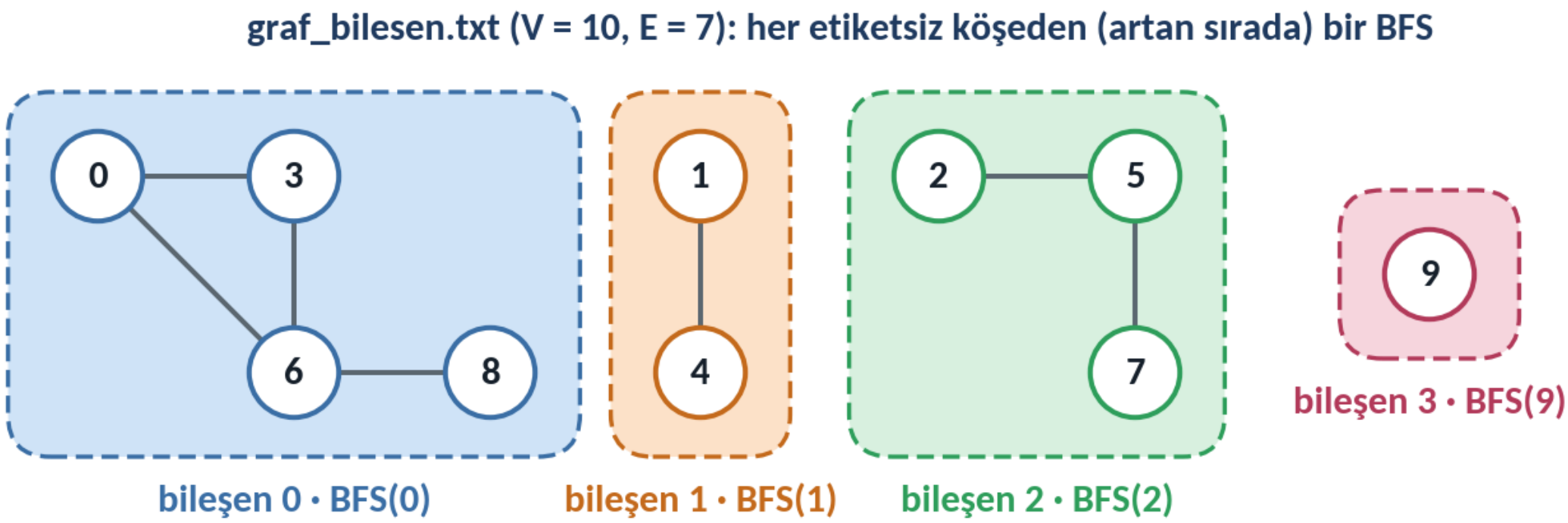
C: tepedeki çerçeve sıradaki komşuya ilerler (Örnek 6)

c/H11_06_dfs_acik_yigin.c · satır 294–308

C

```c
    while (n > 0) {
        Cerceve *c = &y[n - 1];
        if (c->i < g->komsu[c->u].boyut) {
            int v = g->komsu[c->u].veri[c->i++];
            if (g->yonlu) {
                d->tur[d->tur_sayisi++] = sinif(d, c->u, v);
            }
            if (d->renk[v] == BEYAZ) {
                kesfet(d, v);
                y[n++] = (Cerceve){v, 0};
                if (iz) {
                    printf("   it %d          ", v);
                    cerceve_yaz(y, n);
                }
            }
```

Bağlı bileşenler: her beyaz köşeden bir dolaşma

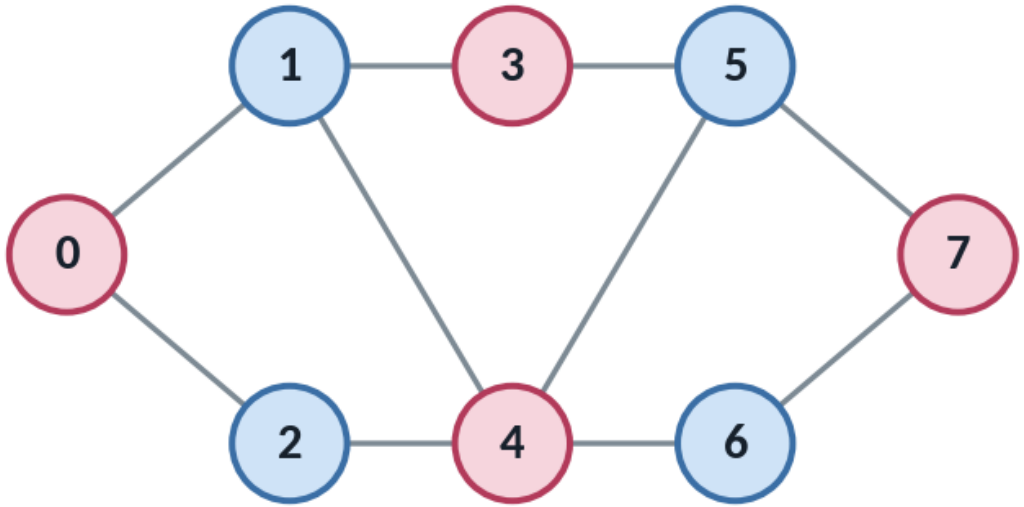



```
etiket = [0 1 2 0 1 2 0 2 0 3] · bileşen sayısı = 4
```

V = 10, E = 7: bileşen sayısı = 4 · BFS çağrısı = 4, kuyruğa ekleme = 10 (= V), kenar inceleme = 14 (= 2E)

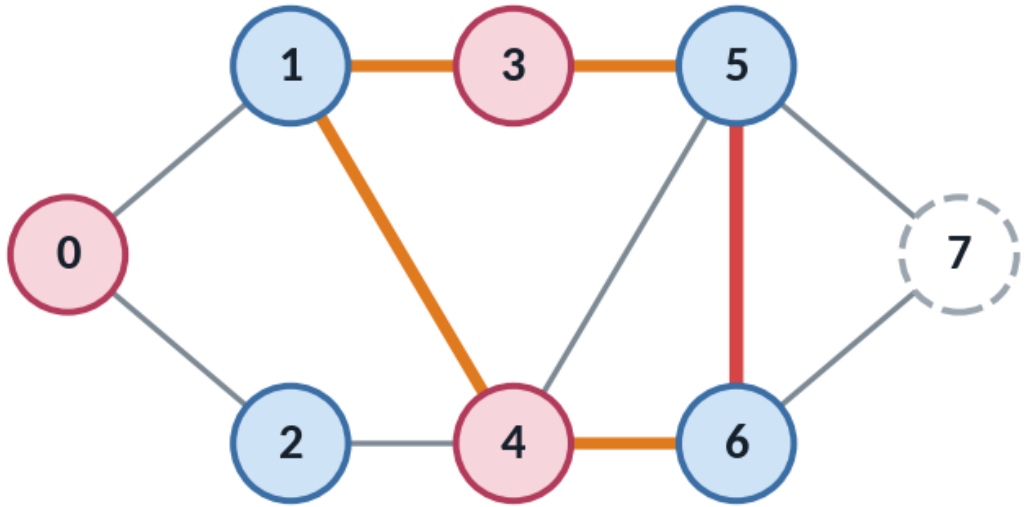
İki parçalılık: BFS ile 2-renklendirme

(a) G: iki parçalı



A = {0 3 4 7} · B = {1 2 5 6}

(b) G + kenar 5–6: değil



kırmızı: çelişkili kenar (5, 6) — iki ucu aynı renk

turuncu: tek uzunluklu döngü 5 → 3 → 1 → 4 → 6 → 5

G: iki parçalı: evet; A = {0 3 4 7}, B = {1 2 5 6} · 5–6 eklenince: tek uzunluklu döngü: 5 → 3 → 1 → 4 → 6 → 5

Örtük graf: ızgarada taşkın doldurma ve ada sayma

(a) Taşkın doldurma: (0, 0)'dan 24 dalga

0	1			0	1	2	3	4	5	6	7		3
1				9	0		4	5	6	7	8		2
2	3		7	8				6	7		9	0	1
3	4	5	6	7		?		5	6			9	0
		6	7	8				4	5	6	7	8	9
1		7	8	9	0	1	2	3	4	5			0
0	9	8			1	2		4	5		?		1
1	0	9		3	2		6	5	6	7			2

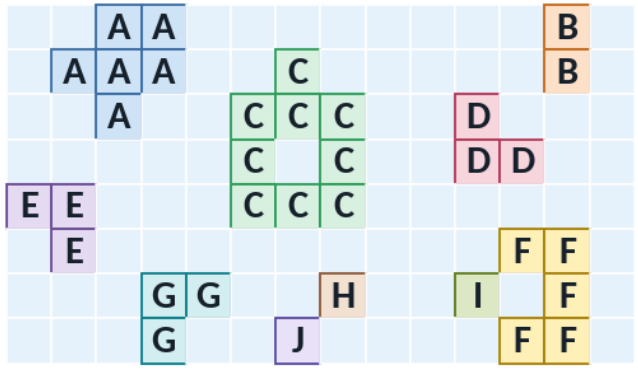
sayı: dalga numarası (mod 10); ? = göl

doldurulan 76 hücre, 24 dalga

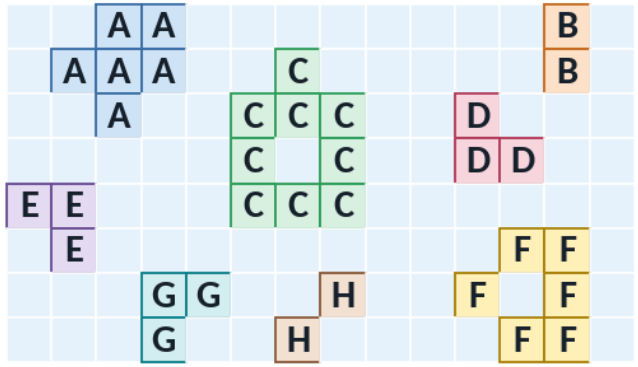
komşu denemesi 304; sınır dışı 36

(sınır dışı: diziye erişmeden reddedildi)

(b1) Adalar, 4-komşuluk: 10 ada



(b2) Adalar, 8-komşuluk: 8 ada



Doldurulan 76 hücre, 24 dalga · Ada sayma, 4-komşuluk (yalnız dik yönler): 10 ada ↔ 8-komşuluk (çaprazlar da): 8 ada

TAHMİN ET

Açık oda 5 × 9 (duvarsız iç), başlangıç (1, 1), çıkış (3, 1): BFS 2 adımlık yolu bulur. H3'ün geri izlemesi (sırayla sağ, aşağı, sol, yukarı dener) ilk bulduğu yolda kaç

A 2: geri izleme de en kısa yolu bulur

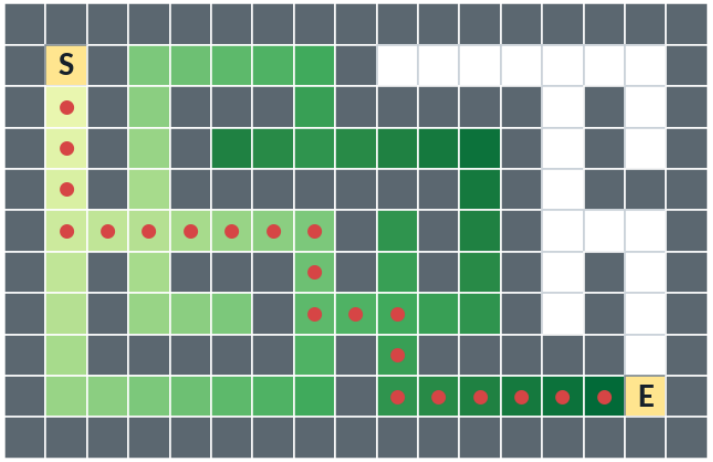
B 14: önce sağa gider, ilk bulduğu yolu döndürür

C 4: bir kez sapar, geri döner

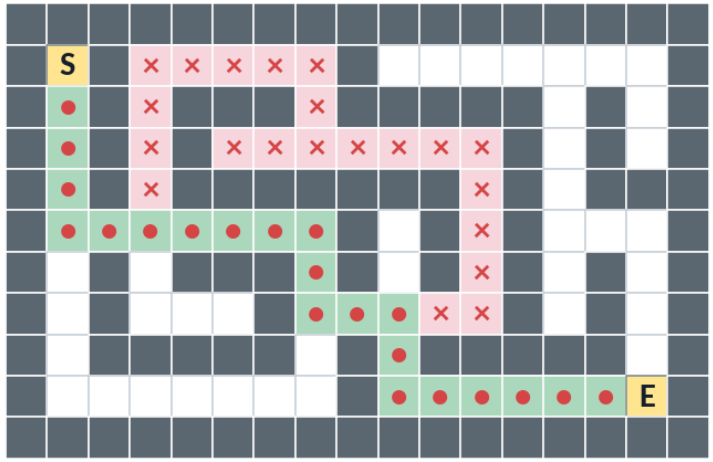
D 20: bütün açık hücrelerden geçer

Labirentte en kısa yol: BFS katmanları ↔ H3 geri izlemesi

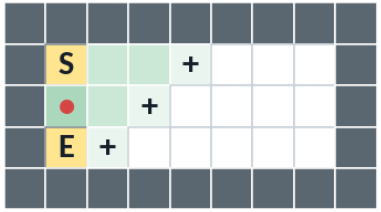
(a) BFS: yol 22 adım, işlenen 61



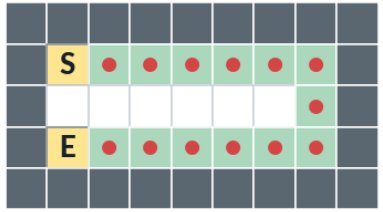
(b) Geri izleme (H3): yol 22, çağrı 44



(c) Açık oda, BFS: yol 2, işlenen 6



(d) Açık oda, geri izleme: yol 14, çağrı 15



renk: BFS katmanı (koyu = uzak) · ● yol · × çıkmazdan geri dönülen · + kuyrukta bekleyen

BFS en kısa yolu her zaman bulur; geri izleme (DFS) ilk bulduğu yolda durur

H3 labirenti: BFS 22 adım, 61 hücre işler · geri izleme 22 adım, 44 çağrı · açık odada 2 ↔ 14 adım

“Kaç adım uzakta?”: ortak yazarlık ağında BFS



en uzak (5 adım) Lale: Erdem → Ada → Deniz → Hakan → Jale → Lale

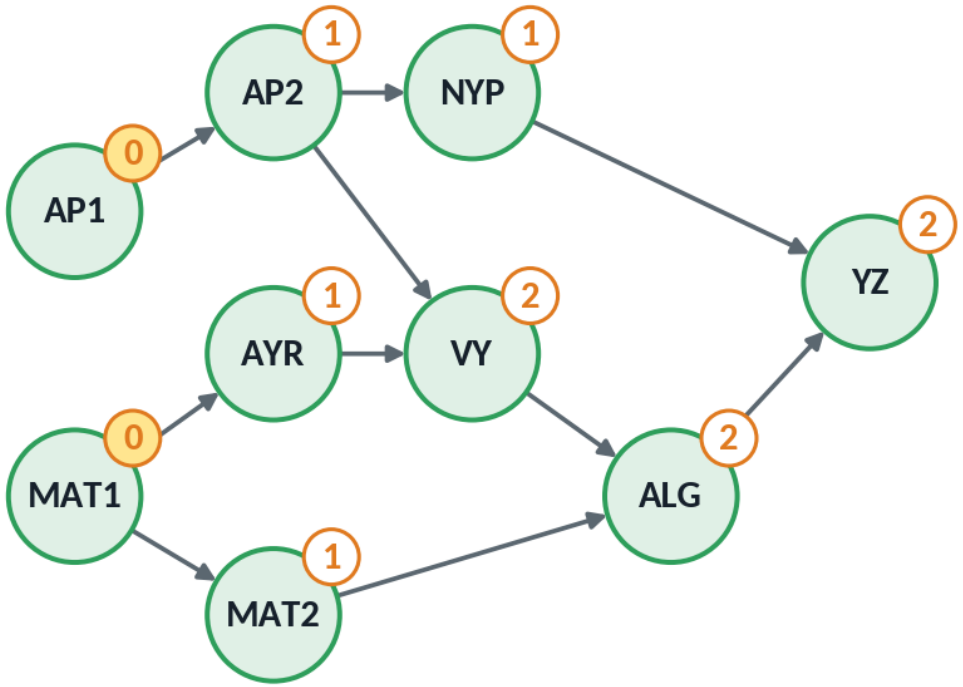
KURGUSAL ağ (Örnek 13): sütun = Erdem'den BFS uzaklığı; ∞ = zincir yok

- Aynı makaledeki iki yazar arasında kenar; ad → köşe **ikili aramayla** (H4).
- En uzak: Lale (5 adım).
- **Erdős sayısı** (Grossman, 2025): 1 sayılı 504, 2 sayılı 6 593 kişi; ortanca 5, ortalama 4,65.
- Yapay 2000 kişilik ağ: ulaşılan 1960 / 2000, ortalama uzaklık = 1,78, en büyük = 4.

Kurgusal ağ: 16 makale, 19 yazar; 4 kişi ulaşılamaz (∞)

Kahn: giriş derecesi 0 olan dersi al, ardıllarını azalt

Ders DAG'ı (dersler_dag.txt)



turuncu rozet: başlangıçtaki giriş derecesi

`sıra: AP1 AP2 MAT1 AYR MAT2 NYP VY ALG YZ`

Kahn: en küçük öbekten çıkar

adım	çıkan	öbek (sonra)
0	–	AP1 MAT1
1	AP1	AP2 MAT1
2	AP2	MAT1 NYP
3	MAT1	AYR MAT2 NYP
4	AYR	MAT2 NYP VY
5	MAT2	NYP VY
6	NYP	VY
7	VY	ALG
8	ALG	YZ
9	YZ	boş

Giriş dereceleri ALG:2 AP1:0 AP2:1 AYR:1 MAT1:0 MAT2:1 NYP:1 VY:2 YZ:2 · öbek: en küçük numara önce

TAHMİN ET

Kahn (en küçük öbek): AP1 alındı, AP2'nin derecesi 0'a indi. Öbek {AP2 (2), MAT1 (4)}. Sıradaki ders hangisi?

A AP2 (2): öbekteki en küçük numara

B MAT1 (4): öbeğe ondan önce girdi

C NYP (6): AP2'nin ardılı

D ALG (0): numarası en küçük ders

Kahn: sözde kod, varlık koşulu ve en erken dönem

```
1  algoritma KAHN(G)
2    her v: derece[v] ← giriş derecesi; dönem[v] ← 1
3    H ← { v : derece[v] = 0 }      ▷ en küçük öbek
4    iken H boş değil
5      u ← EN-KÜÇÜĞÜ-ÇIKAR(H); sıra'ya u ekle
6      u'nun her v ardılı için
7        dönem[v] ← max(dönem[v], dönem[u] + 1)
8        derece[v] ← derece[v] − 1
9        eğer derece[v] = 0 ise EKLE(H, v)
10   eğer |sıra| < V ise döndür DÖNGÜ VAR
11   döndür sıra, dönem
```

- **Varlık:** topolojik sıra ⇔ graf DAG. Döngüdeki derslerin derecesi hiç 0'a inmez.
- **Döngülü katalog:** yalnız AP1 MAT1 AYR MAT2 (4 / 9) sıralanır.
- **En erken dönem** = en uzun önkoşul zinciri; BFS katmanı en kısa zinciri verir (farklı!).
- **Θ(V + E)** (FIFO); öbekle Θ(E + V log V).

Python: Kahn'ın ana döngüsü (Örnek 11)

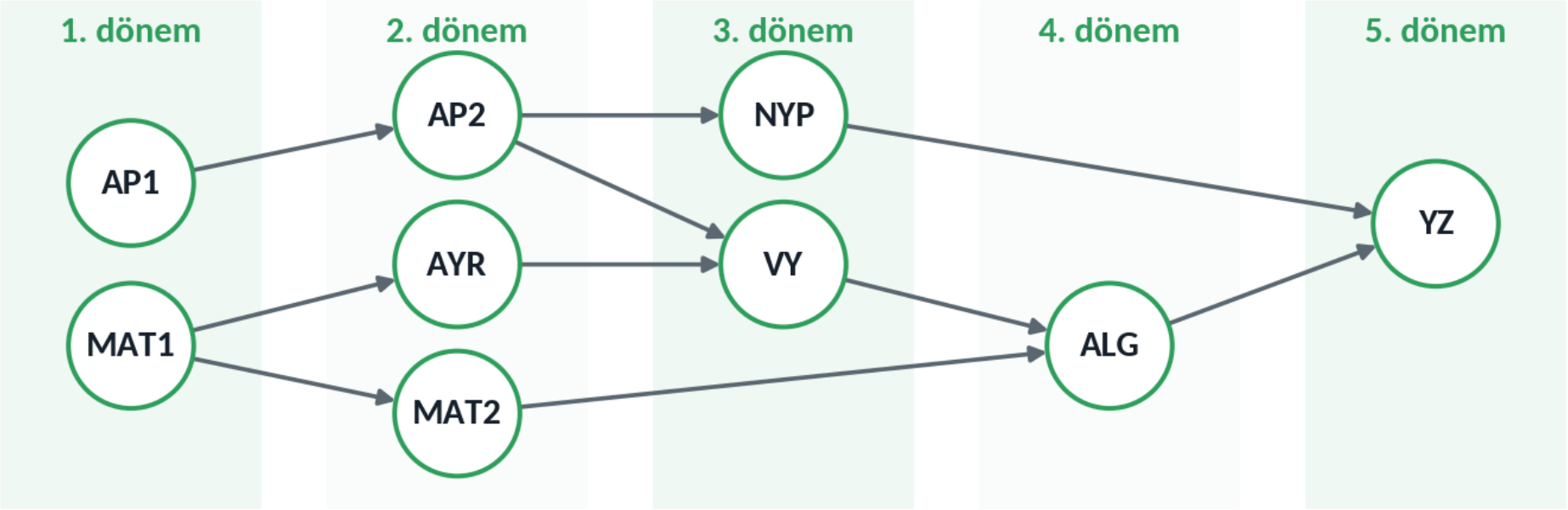
python/H11_11_kahn.py · satır 142–155 — Python

```python
    while (q if fifo else h):
        u = q.popleft() if fifo else heapq.heappop(h)
        sira.append(u)
        parcalar = []
        for v in g.komsu[u]:
            donem[v] = max(donem[v], donem[u] + 1)  # en uzun zincir
            derece[v] -= 1
            parcalar.append(f" {v}: {derece[v] + 1}->{derece[v]}"
                            + (" (öbeğe)" if derece[v] == 0 else ""))
            if derece[v] == 0:
                if fifo:
                    q.append(v)
                else:
                    heapq.heappush(h, v)
```

ÖRNEK 11 · DERS ÖNKOŞUL GRAFI (DAG)

```
topolojik sıra (en küçük öbek): 1 2 4 3 5 6 7 0 8
en erken dönem: 0:4 1:1 2:2 3:2 4:1 5:2 6:3 7:3 8:5
```

Önkoşul DAG'ı: en erken dönemler ve 108 geçerli sıra



Kahn, en küçük öbek:	AP1 → AP2 → MAT1 → AYR → MAT2 → NYP → VY → ALG → YZ
Kahn, FIFO kuyruk:	AP1 → MAT1 → AP2 → AYR → MAT2 → NYP → VY → ALG → YZ
DFS bitişinin tersi:	MAT1 → MAT2 → AYR → AP1 → AP2 → VY → NYP → ALG → YZ

Bu DAG'ın geçerli topolojik sıra sayısı: 108 (üçü yukarıda)

dönem 1: AP1 MAT1 · dönem 2: AP2 AYR MAT2 · dönem 3: NYP VY · dönem 4: ALG · dönem 5: YZ

Üç dilde: giriş derecelerini say (Kahn'ın ilk geçişi)

H11_11_kahn.c — C

```c
for (int u = 0; u < g->V; u++) {
    for (size_t i = 0; i < g->komsu[u].boyut; i++) {
        derece[g->komsu[u].veri[i]]++;
    }
}
```

H11_11_kahn.cpp — C++

```cpp
for (int u = 0; u < g.V; u++) {
    for (int v : g.komsu[u]) {
        derece[v]++;
    }
}
```

H11_11_kahn.py — Python

```python
for u in range(g.V):
    for v in g.komsu[u]:
        derece[v] += 1
```

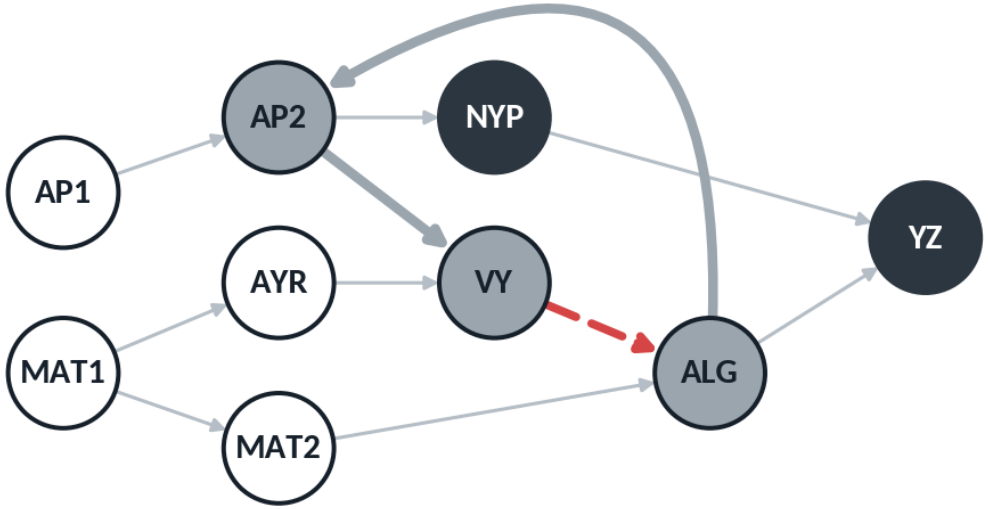
Her kenar u → v, v'nin giriş derecesini 1 artırır: Θ(V + E); Python'da [0] * V ve iç içe for (H11_11_kahn; satırlar: C 265–269 · C++ 180–184 · Python 128–130)

DFS ile topolojik sıra: bitiş sırasının tersi; gri komşu = döngü

(a) DAG: DFS bitiş sırası ve tersi

bitiş (siyaha dönüş):	YZ	ALG	NYP	VY	AP2	AP1	AYR	MAT2	MAT1
tersi = topolojik sıra:	MAT1	MAT2	AYR	AP1	AP2	VY	NYP	ALG	YZ

(b) Hatalı önkoşul ALG → AP2 eklenince



DFS(ALG): gri = yolda, siyah = bitti

kalın gri: gri yol

kesikli: VY → ALG gri köşeye gidiyor = geri kenar

döngü:

ALG → AP2 → VY → ALG

DAG'da u → v için v gri olamaz: bitiş[v] < bitiş[u]

(a) sıra MAT1 MAT2 AYR AP1 AP2 VY NYP ALG YZ · (b) döngü ALG → AP2 → VY → ALG

TAHMİN ET

Döngülü katalogda (ALG → AP2 eklendi) DFS-TOPO kökler artan sırada başlar: ALG (0), sonra AP2, NYP, YZ… Döngüyü hangi kenarda fark eder?

A VY → ALG: ALG o anda GRİ (geri kenar)

B ALG → AP2: döngüyü kapatan, yeni eklenen kenar

C AP2 → NYP: NYP'nin ardılı YZ zaten görülmüş

D NYP → YZ: YZ siyah

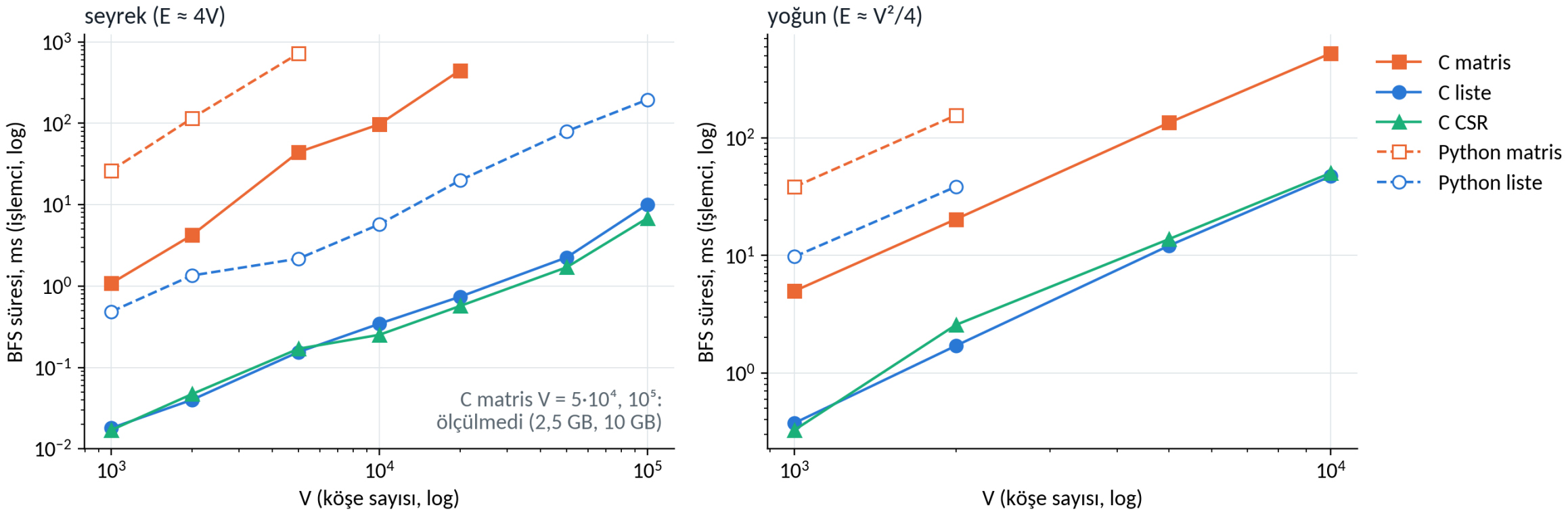
ALGORİTMA KİMLİK KARTI

Topolojik sıralama (Kahn / DFS) *topological sort (Kahn's algorithm / DFS)*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Kahn: derece 0 → sıra; DFS: bitişin tersi	Yönlü graf; yalnız DAG'da sıra var	Θ(V + E)	Θ(V + E)	Θ(V + E)	O(V)

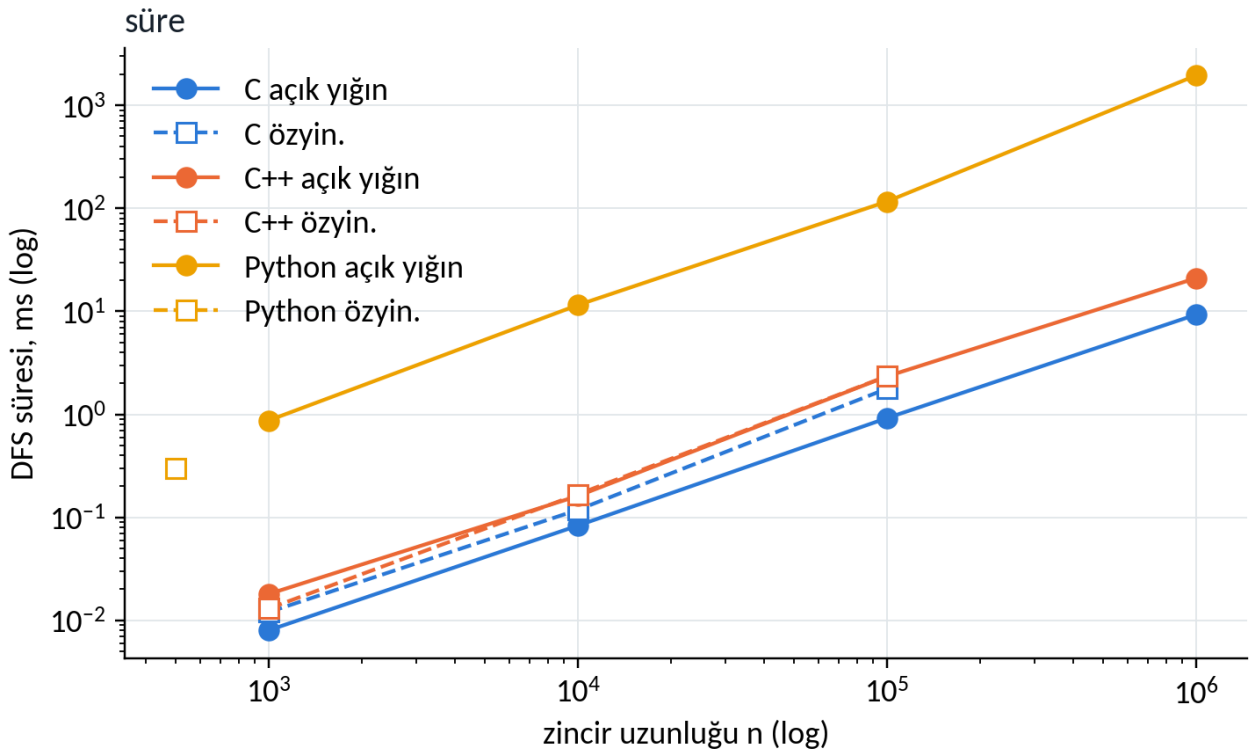
Özellikler	Birden çok geçerli sıra: ders DAG'ında 108; en küçük öbek belirli bir sıra seçer
Değişmez / doğruluk	Sıradaki her köşenin bütün öncülleri daha önce; DFS'de bitiş[v] < bitiş[u]
Kütüphane karşılığı	C: elle · C++: Boost.Graph topological_sort · Python: graphlib (3.9+), networkx (kâhin)
Ne zaman kullanılır	Önkoşul/bağımlılık sırası, derleme sırası, görev planlama
Ne zaman kullanılmaz	Döngülü graf (önce döngüyü raporla); yönsüz graf

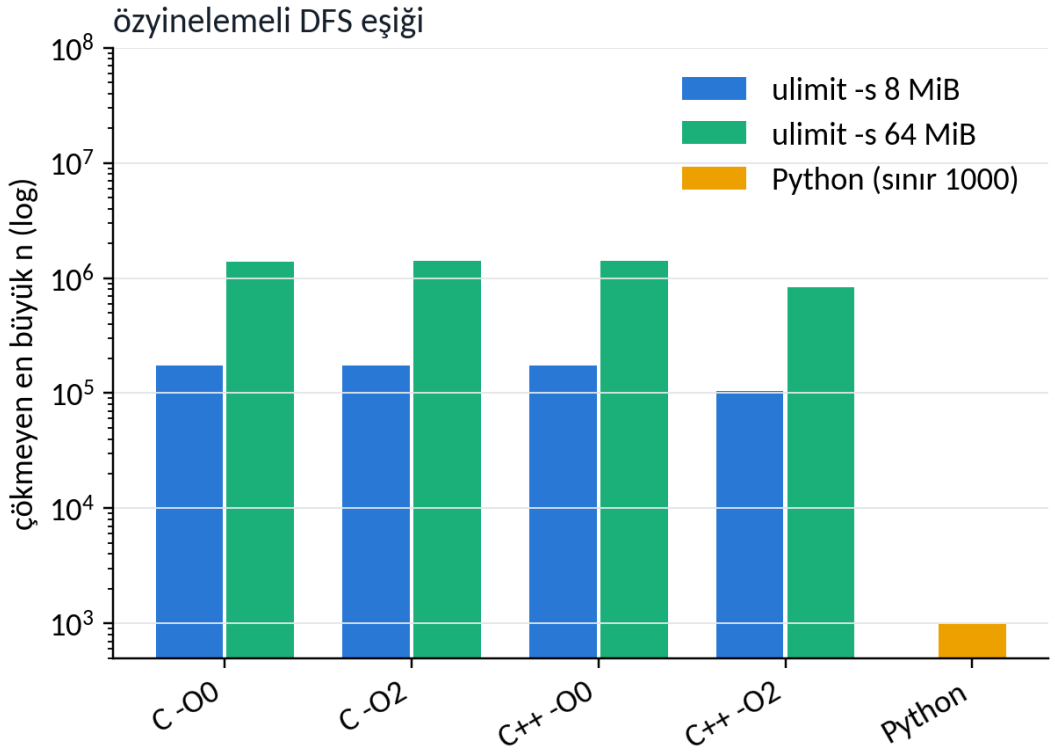
Haftanın Deneyi (a): matris ↔ liste ↔ CSR, seyrek ve yoğun



*Seyrek, C: eğim matris 2,006, liste 1,345 · V = 2·10⁴'te matris 447,4 ms ↔ liste 0,74 ms · * kestirim*

Haftanın Deneyi (b): derin zincirde DFS — süre ve çökme eşiği





8 MiB yığın: C -O0 174 061, C++ -O2 104 432 köşede durur; 64 MiB'ta C -O0 1 397 384 · Python 995

Hata Avcısı: yedi hata, bu makinede gözlenen belirtiler

#	Hata	Belirti (gerçek çıktı)	Çözüm
1	İşareti kuyruktan çıkarırken koymak	G'de 16 ekleme (V = 8); ızgarada üstel	İşaret kuyruğa eklerken
2	Yönsüz kenarı tek yönde eklemek (C++)	Σ derece = 10 (2E = 20); BFS(7) kimseye ulaşmaz	Her kenar iki listeye
3	Sınırı komşuya eriştikten sonra denetlemek	ASan: heap-buffer-overflow; Python'da sessizce sondan okur	Önce icinde(ni, nj), sonra erişim
4	Ağırlıksız en kısa yolu DFS ile aramak	2'ye uzaklık 5 (BFS: 1)	BFS
5	Topolojik sıralamada döngüyü denetlememek	eksik plan: AP1 MAT1 MAT2 AYR	|sıra| < V ise döngü; üç renk
6	Python: def dfs(g, u, ziyaret=[])	3. çağrı [0, 1, 3, 2, 0, 4, 5]	ziyaret=None, içeride yeni liste
7	Derin zincirde özyinelemeli DFS	C, n = 10^6: SIGSEGV (çıkış 139); Python: RecursionError	Açık yığın (Örnek 6)

Hata Avcısı 1: işaret çıkarırken konursa kuyruk şişer

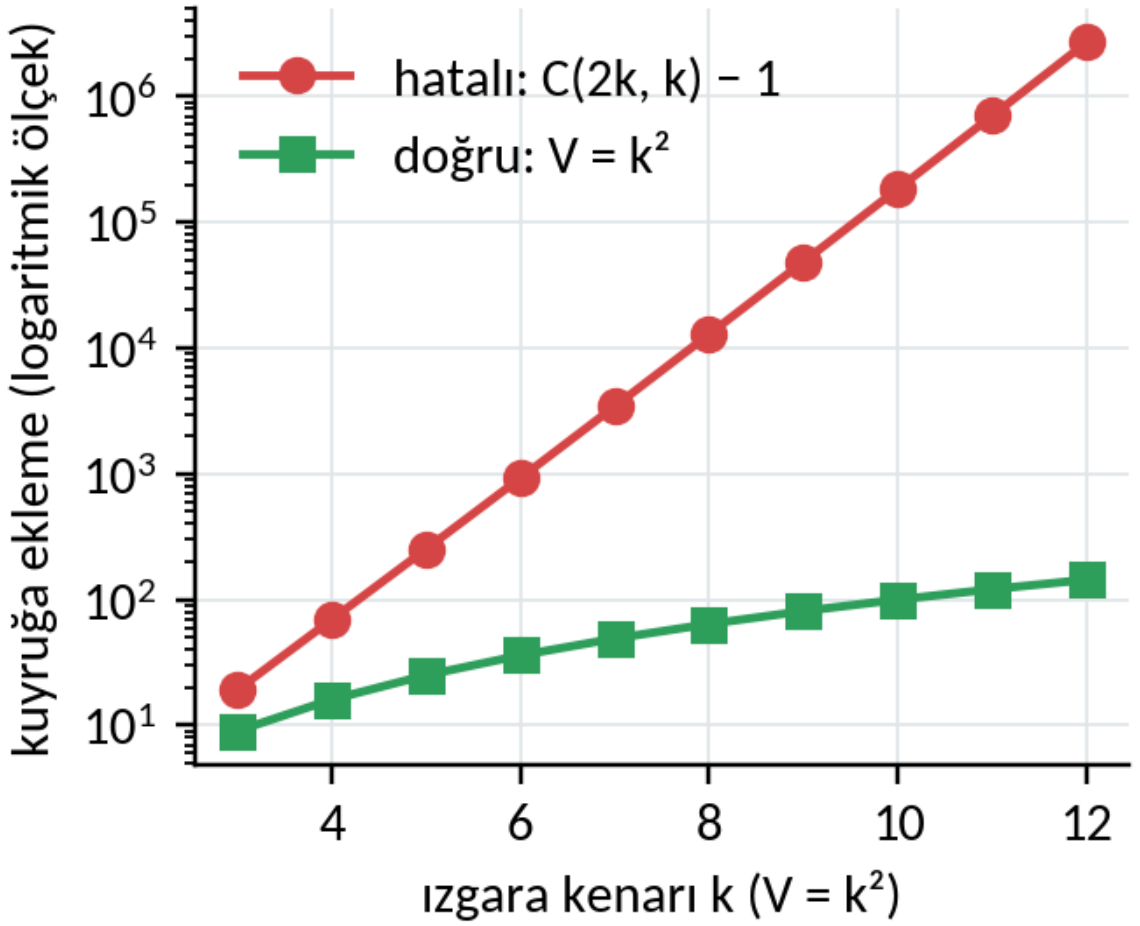
G'de BFS(0): işlenme sırası

hatalı (işaret ÇIKARIRKEN):

0	1	2	3	4	4	5	5
6	5	6	7	7	7	7	7

doğru (işaret EKLERKEN):

0	1	2	3	4	5	6	7



G'de 7 köşesi 5 kez işlenir · k × k ızgarada C(2k, k) − 1 ekleme: k = 12'de (V = 144) 2 704 155

Dil Penceresi: graflar üç dilde

	C	C++	Python
Komşuluk listesi	KomsuDizisi *komsu (≈ int ** + boyut dizisi), realloc ile büyür (H8)	vector<vector<int>>; push_back	liste listesi; adlı köşelerde dict
Komşuluk matrisi	int m[MAKS][MAKS] ya da calloc(V * V, 1)	vector<vector<int>>(V, vector<int>(V))	[[0] * V for _ in range(V)] — [[0] * V] * V satırı paylaşır
Kuyruk (BFS)	dizi kuyruğu (H9)	std::queue<int>	collections.deque: popleft O(1)
Yığın (DFS)	dizi yığın (H9) / çerçeve dizisi	std::vector ya da std::stack	liste: append, pop
Özyineleme sınırı	çağrı yığını, ulimit -s (8 MiB): SIGSEGV	aynı; çerçeve boyu bayrağa bağlı	1000 çağrı: RecursionError
Graf kütüphanesi	yok	standartta yok; Boost.Graph	standartta yok (graphlib 3.9+); networkx yalnız doğrulama, ödevde değil

Mühendis Gözüyle · Atölyeler · İleri Bakış · Merak

Mühendis Gözüyle

- Temsil bir modelleme kararı: seyrekte liste/CSR; v2.10'da V = 16 000'de matrisle Kahn 249 kat yavaş.
- Üretim kodunda derinliği girdiye bağlı özyineleme yok: açık yığın.
- Belirlilik: artan komşu, en küçük numara — test edilebilirlik.

Atölyeler

- İnsan BFS'si: tanışıklık ağında dalga (sınıfta).
- Labirent yarışması: BFS, DFS, geri izleme; ziyaret sayısı.
- Ders planlayıcı (eşli): önkoşuldan DAG ve topolojik sıra.

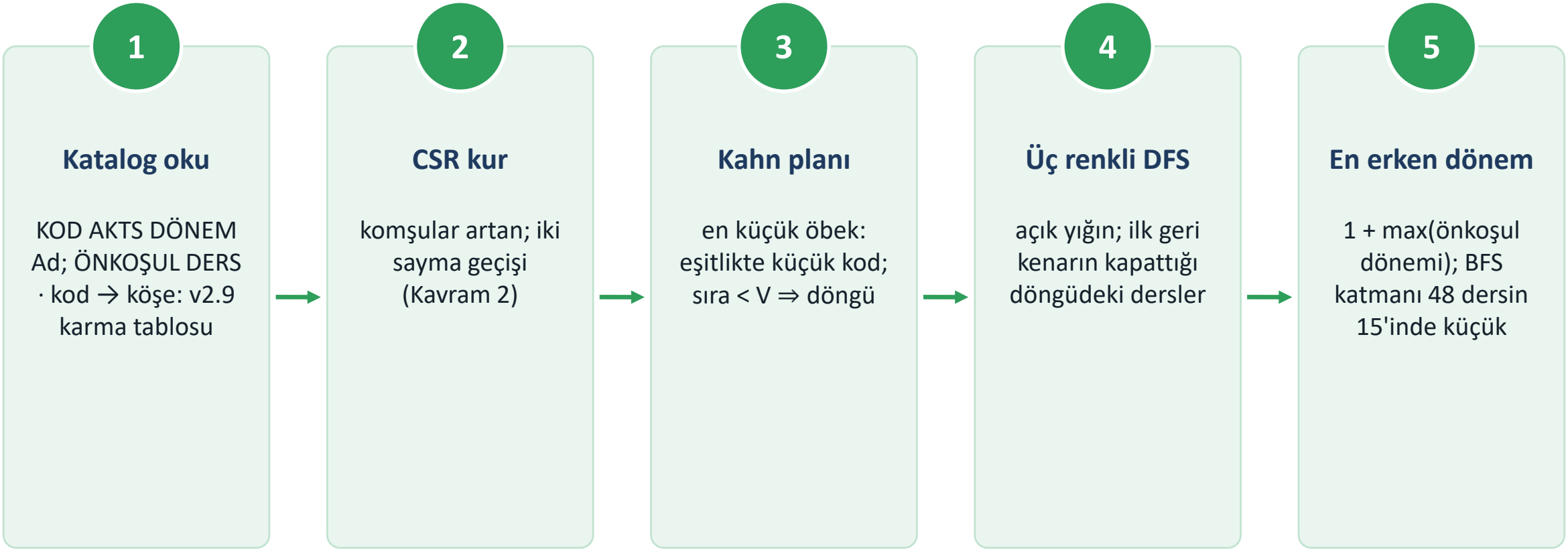
İleri Bakış

- H12: ağırlıklı grafta Dijkstra; Kruskal ve Prim.
- Kuvvetli bağlı bileşenler, köprüler, ağ akışı; graf veritabanları, PageRank.
- **Hazırlık (H12):** Bölüm 4.1 ve 4.7, H7 öbeği, oynatıcılar, Alıştırma 11.1–11.7.

Merak

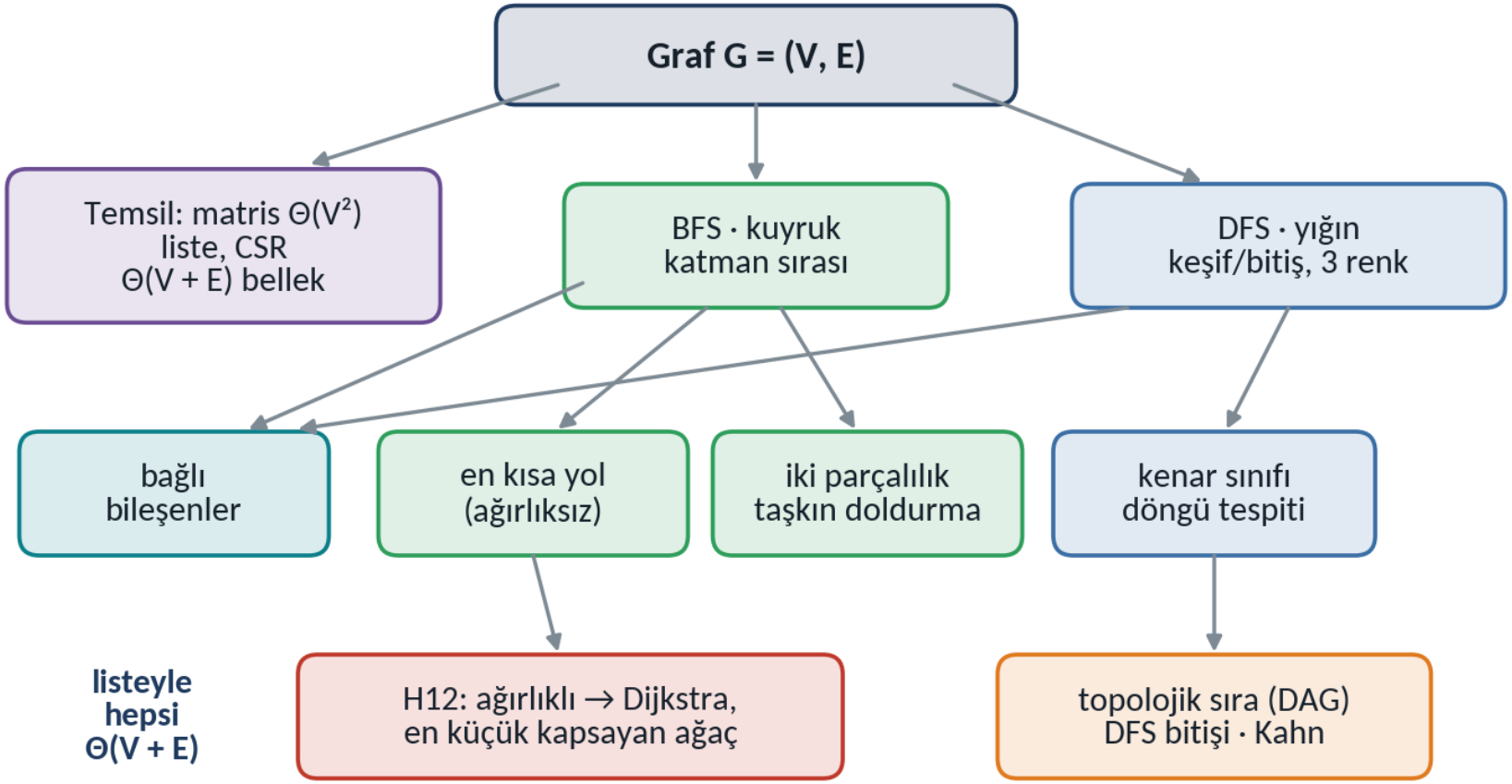
- Euler yolu ⇔ tek dereceli köşe sayısı 0 ya da 2. G'de tek dereceliler 1 ve 5: yolu bulun.
- Königsberg'de tur için kaç köprü eklemek gerekir?

Mini-OBS 2 v2.10: önkoşul grafından ders planına



v2.10 belgesi § 8.1: V = 1 000 000, E = 4 000 000 rastgele DAG'da bütün plan 1,03 s · 8 kabul ölçütü (§ 10)

Özet: temsil, iki dolaşma ve uygulamaları



Temsil (matris Θ(V²), liste/CSR Θ(V + E)) → BFS (kuyruk, katman) ve DFS (yığın, üç renk) → uygulamalar → H12

ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1 G'de (temsil slaytındaki graf) BFS(7) katmanlarını yazın. En uzak köşe hangisi?

2 Döngülü bir önkoşul grafında Kahn ne üretir; döngüdeki dersleri nasıl bulursunuz?

3 Python'da 10^5 köşelik bir zincirde özyinelemeli DFS neden çöker? İki satırda düzeltin.